

IMPLEMENTASI ALGORITMA K-NEAREST NEIGHBORS (KNN) DALAM KLASIFIKASI PERFORMA AKADEMIK SISWA PADA DATASET STUDENT PERFORMANCE

Andre Astamam¹⁾, Baiq Erwina Yolanda²⁾, Fauzan Hari Ramdani³⁾, Lalu Galang Abdullah.⁴⁾ Program Studi Teknik Informatika Universitas Mataram

Program Studi Teknik Informatika Universitas Mataram

Corresponding author e-mail: evisetvx@gmail.com, baigerwinayolanda26@gmail.com,
fauzanhari4@gmail.com, abdullahgalang06@gmail.com

Abstrak

Performa akademik siswa merupakan salah satu indikator penting dalam menganalisis perkembangan dan potensi siswa, namun pengolahan data berskala besar secara sekuensial sering menyebabkan waktu eksekusi yang tidak efisien. Penelitian ini menerapkan algoritma K-Nearest Neighbors (KNN) yang dipadukan dengan komputasi paralel menggunakan Message Passing Interface (MPI) untuk meningkatkan efisiensi klasifikasi. Dataset yang digunakan terdiri dari 300.000 data latih dan 75.000 data uji, diproses secara terdistribusi pada beberapa core melalui mekanisme pembagian partisi MPI. Hasil percobaan menunjukkan bahwa variasi jumlah core (N) mencapai waktu eksekusi terbaik pada N=3 sebesar 224,84 detik, namun pada N selanjutnya waktu eksekusi kembali meningkat hingga 312,57 detik pada N=7, yang mengindikasikan adanya overhead komunikasi antar-proses seiring bertambahnya jumlah core. Pada variasi nilai K, akurasi meningkat dari 96,11% pada K=1 hingga 96,82% pada K=9, menandakan kestabilan prediksi yang lebih baik pada nilai K yang lebih besar. Variasi jumlah dimensi fitur menunjukkan waktu eksekusi meningkat dari 15,34 detik pada 4 dimensi hingga 30,84 detik pada 8 dimensi, sedangkan variasi jumlah data latih memperlihatkan peningkatan waktu dari 48,36 detik pada 100.000 data hingga 224,81 detik pada 320.000 data. Penelitian ini membuktikan bahwa kombinasi KNN dan MPI efektif dalam menangani data berskala besar dengan performa klasifikasi yang stabil.

Kata kunci: K-Nearest Neighbors, Message Passing Interface, komputasi paralel, klasifikasi, waktu eksekusi.

Abstract

Student academic performance is an important indicator for analyzing student development and potential; however, processing large-scale data sequentially often leads to inefficient execution times. This study applies the K-Nearest Neighbors (KNN) algorithm combined with parallel computing using Message Passing Interface (MPI) to improve classification efficiency. The dataset consists of 300,000 training data and 75,000 test data, processed in a distributed manner across multiple cores through MPI's partition mechanism. The experimental results show that the variation in the number of cores (N) achieves the best execution time at N=3 with 224.84 seconds; however, execution time increases again in subsequent values, reaching 312.57 seconds at N=7, indicating communication overhead between processes as the number of cores grows. In the K value variation, accuracy improves from 96.11% at K=1 to 96.82% at K=9, reflecting greater prediction stability at higher K values. The feature dimension variation shows execution time increasing from 15.34 seconds at 4 dimensions to 30.84 seconds at 8 dimensions, while the training data variation reveals a proportional increase from 48.36 seconds at 100,000 data points to 224.81 seconds at 320,000 data points. This study proves that the combination of KNN and MPI is effective in handling large-scale data with stable classification performance.

Keyword: K-Nearest Neighbors, Message Passing Interface, parallel computing, classification, execution time.

I. PENDAHULUAN

Perkembangan teknologi informasi telah mendorong institusi pendidikan untuk menghasilkan data siswa dalam jumlah yang semakin besar. Data tersebut umumnya mencakup nilai akademik, jumlah kehadiran, serta aktivitas belajar siswa [1]. Meskipun data tersebut berpotensi memberikan wawasan mendalam mengenai perkembangan siswa, dalam praktiknya data tersebut sering kali belum dimanfaatkan secara optimal untuk keperluan analisis lanjutan [2].

Proses analisis secara manual menjadi kurang efektif seiring meningkatnya volume data. Banyaknya atribut dalam data siswa turut membuat evaluasi menjadi lebih kompleks, sehingga menyulitkan identifikasi pola yang dapat digunakan sebagai dasar pengambilan keputusan [3]. Oleh karena itu, diperlukan pendekatan yang mampu mengolah data secara otomatis dan sistematis, salah satunya adalah teknik klasifikasi dalam data mining [1].

Algoritma K-Nearest Neighbors (KNN) merupakan salah satu metode klasifikasi yang sederhana namun efektif [3]. Beberapa penelitian menunjukkan bahwa KNN mampu memberikan hasil klasifikasi yang baik pada data pendidikan dengan tingkat akurasi yang cukup tinggi [4]. Namun, seiring meningkatnya jumlah data, proses perhitungan jarak pada KNN dapat menjadi lebih lambat [5]. Untuk mengatasi kelemahan tersebut, diperlukan pendekatan komputasi paralel menggunakan Message Passing Interface (MPI), yang memungkinkan pembagian beban komputasi ke beberapa proses sehingga dapat mempercepat waktu eksekusi [6].

Penelitian ini bertujuan untuk mengimplementasikan algoritma KNN dalam mengklasifikasikan performa akademik siswa menggunakan dataset Student Performance, menganalisis tingkat akurasi yang dihasilkan, mengimplementasikan komputasi paralel dengan MPI pada algoritma KNN, dan membandingkan efisiensi waktu eksekusi antara pendekatan sekuensial dan paralel.

II. TINJAUAN PUSTAKA

A. Data Mining

Data mining merupakan proses penggalian informasi dari sekumpulan data berukuran besar untuk menemukan pola atau hubungan yang sebelumnya tidak diketahui [7]. Proses ini memanfaatkan berbagai teknik dari bidang statistik, *machine learning*, dan basis data untuk menghasilkan informasi yang berguna dalam

pengambilan keputusan. Dalam bidang pendidikan, data mining dapat digunakan untuk menganalisis performa siswa berdasarkan berbagai faktor seperti nilai, kehadiran, dan partisipasi dalam kelas [8].

B. Klasifikasi

Klasifikasi merupakan salah satu teknik dalam data mining yang digunakan untuk mengelompokkan data ke dalam kategori tertentu berdasarkan karakteristik yang dimilikinya [9]. Proses klasifikasi dilakukan dengan menggunakan data latih yang telah memiliki label, kemudian model yang dihasilkan digunakan untuk memprediksi kelas dari data baru [10]. Dalam penelitian ini, klasifikasi digunakan untuk mengelompokkan siswa ke dalam kategori nilai A, B, C, D, dan F berdasarkan data akademik yang tersedia.

C. Algoritma K-Nearest Neighbor (KNN)

K-Nearest Neighbor (KNN) merupakan algoritma klasifikasi yang bekerja berdasarkan konsep kedekatan jarak antar data [11]. Algoritma ini mengklasifikasikan data baru dengan melihat sejumlah K data terdekat (*neighbor*), kemudian menentukan kelas berdasarkan mayoritas dari tetangga terdekat tersebut. KNN termasuk dalam metode non-parametrik dan sering disebut sebagai *lazy learning* karena tidak melakukan proses pelatihan model secara eksplisit. Proses perhitungan dilakukan saat tahap prediksi dengan menghitung jarak antara data uji dan seluruh data latih menggunakan Euclidean Distance [12].

Rumus Euclidean Distance yang digunakan adalah sebagai berikut:

$$d(x,y) = \sqrt{\sum_i (x_i - y_i)^2} \quad (1)$$

Kelemahan utama KNN adalah waktu komputasi yang meningkat seiring bertambahnya jumlah data, karena setiap data uji harus dibandingkan dengan seluruh data latih [12]. Performa KNN juga sangat dipengaruhi oleh pemilihan nilai K dan skala data yang digunakan.

D. Komputasi Paralel dan MPI

Komputasi paralel merupakan teknik pemrosesan data dengan cara membagi suatu pekerjaan menjadi beberapa bagian yang dapat dijalankan secara bersamaan [13]. Pendekatan ini digunakan untuk meningkatkan efisiensi dan mempercepat waktu eksekusi program, terutama pada

pengolahan data dalam jumlah besar. Message Passing Interface (MPI) merupakan standar komunikasi yang digunakan dalam pemrograman paralel pada sistem terdistribusi, yang memungkinkan beberapa proses untuk saling berkomunikasi dan bertukar data selama eksekusi berlangsung [10]. MPI menyediakan fungsi komunikasi seperti broadcast, scatter, dan gather yang sangat relevan dalam implementasi KNN paralel [14].

III. METODOLOGI PENELITIAN

Penelitian ini menggunakan metodologi yang terdiri dari beberapa tahap, mulai dari pengumpulan data hingga evaluasi model.

A. Pengumpulan dan Pemahaman Data

Dataset yang digunakan adalah data performa siswa yang terdiri dari 10.000 data. Dataset ini memiliki empat atribut fitur, yaitu `weekly_self_study_hours`, `attendance_percentage`, `class_participation`, dan `total_score`, serta `grade` sebagai label kelas. Klasifikasi dilakukan ke dalam lima kategori nilai: A, B, C, D, dan F. Pemilihan dataset ini didasarkan pada relevansinya terhadap permasalahan klasifikasi dalam bidang pendidikan dan sifat numeriknya yang sesuai untuk diolah menggunakan KNN.

B. Pra-pemrosesan Data

Sebelum proses klasifikasi, data melalui tahap pra-pemrosesan yang meliputi penghapusan atribut yang tidak relevan (`student_id`) dan normalisasi data menggunakan metode Min-Max Scaling. Normalisasi penting dilakukan karena algoritma KNN sangat sensitif terhadap perbedaan skala data. Tanpa normalisasi, fitur dengan nilai besar dapat mendominasi perhitungan jarak dan menyebabkan hasil klasifikasi yang tidak akurat.

C. Pembagian Data dan Implementasi KNN

Dataset dibagi menjadi data latih dan data uji dengan rasio 80:20, di mana 80% data digunakan untuk melatih model dan 20% sisanya untuk pengujian. Algoritma KNN diimplementasikan dengan menentukan nilai K, menghitung Euclidean Distance antara data uji dan seluruh data latih, mengurutkan jarak, dan menentukan kelas berdasarkan mayoritas K tetangga terdekat.

D. Implementasi Komputasi Paralel dengan MPI

Untuk meningkatkan efisiensi komputasi, algoritma KNN diimplementasikan menggunakan pendekatan paralel dengan library `mpi4py`. Proses utama (rank 0) bertugas membaca data dan mendistribusikannya ke seluruh proses

menggunakan mekanisme broadcast untuk data latih dan scatter untuk data uji. Setiap proses secara independen menghitung prediksi KNN untuk bagian data ujinya masing-masing. Hasil dari setiap proses dikumpulkan kembali ke proses utama menggunakan operasi reduce untuk dihitung akurasi dan waktu eksekusi akhirnya.

E. Evaluasi Model

Evaluasi dilakukan dengan mengukur waktu eksekusi program menggunakan fungsi waktu pada MPI untuk membandingkan performa antara pendekatan sekuensial dan paralel. Pengujian dilakukan dalam tiga skenario: (1) variasi jumlah core N dengan data tetap, (2) variasi jumlah data latih dengan N=3 tetap, dan (3) variasi nilai K dengan N dan data tetap.

IV. HASIL DAN PEMBAHASAN

A. Pengaruh Jumlah Core (N) terhadap Waktu Eksekusi

Pengujian pertama dilakukan dengan memvariasikan jumlah core (N) dari 1 hingga 7 menggunakan data latih sebanyak 300.000 dan data uji 75.000 dengan nilai K=6. Hasil pengujian disajikan pada Tabel 1.

Tabel 1. Hasil Pengujian Variasi Jumlah Core (N)

Jumlah (Core)	N	Waktu Eksekusi (detik)
1		330,62
2		238,16
3		224,84
4		259,01
5		279,58
6		299,01
7		312,57



Berdasarkan Tabel 1, terdapat pola penurunan waktu eksekusi yang terjadi saat jumlah core ditingkatkan dari N=1 hingga N=3, di mana waktu eksekusi berkurang dari 330,62 detik menjadi 224,84 detik. Penurunan waktu yang paling signifikan

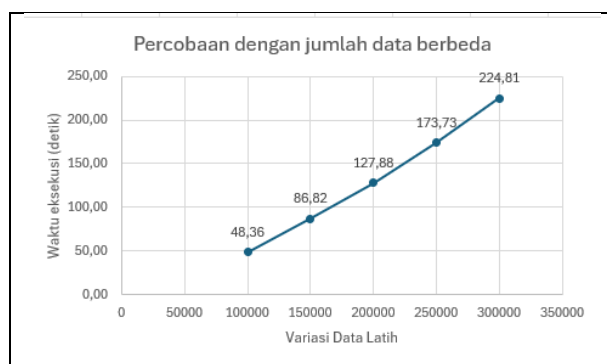
terjadi saat N ditingkatkan dari 1 ke 2, yakni dari 330,62 detik menjadi 238,16 detik. Waktu eksekusi terbaik dicapai pada N=3 sebesar 224,84 detik. Namun, seiring bertambahnya jumlah core melampaui N=3, waktu eksekusi justru kembali meningkat secara bertahap dari 259,01 detik pada N=4 hingga mencapai 312,57 detik pada N=7. Peningkatan ini mengindikasikan adanya overhead komunikasi antar-proses yang semakin besar seiring bertambahnya jumlah core, sehingga manfaat paralelisme tidak lagi sebanding dengan biaya koordinasi yang ditimbulkan. Hal ini mencerminkan prinsip diminishing returns dalam komputasi paralel, di mana penambahan unit kerja di luar titik optimal justru menurunkan efisiensi keseluruhan sistem.

B. Pengaruh Jumlah Data Latih terhadap Waktu Eksekusi

Pengujian kedua dilakukan dengan N=6 core tetap dan memvariasikan jumlah data latih dari 100.000 hingga 300.000 dengan nilai K=6. Hasil pengujian disajikan pada Tabel 2.

Tabel 2. Hasil Pengujian Variasi Jumlah Data Latih (N=6)

Jumlah Data Latih	Waktu Eksekusi (detik)
100000	48,36
150000	86,82
200000	127,88
250000	173,73
300000	224,81



Berdasarkan Tabel 2, terdapat hubungan berbanding lurus antara jumlah data latih dengan waktu eksekusi. Waktu eksekusi meningkat secara konsisten dari 48,36 detik pada 100.000 data latih hingga 224,81 detik pada 300.000 data latih. Peningkatan ini terjadi secara bertahap, yakni 86,82 detik pada 150.000 data, 127,88 detik pada 200.000 data, dan 173,73 detik pada 250.000 data. Hal ini wajar terjadi karena meskipun program dijalankan secara paralel, setiap core tetap harus memproses bagian data yang ukurannya semakin

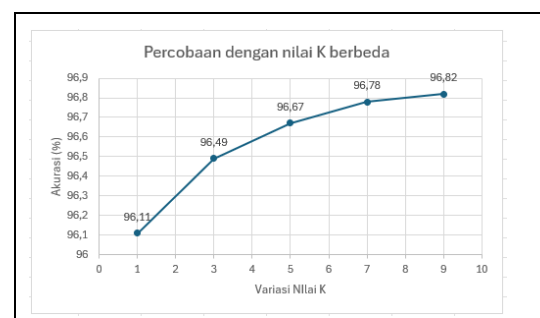
besar seiring bertambahnya jumlah data latih. Bertambahnya jumlah data juga meningkatkan overhead komunikasi antar proses untuk sinkronisasi hasil akhir, sehingga peningkatan waktu eksekusi tidak dapat dihindari. Untuk menangani data yang sangat masif, diperlukan strategi tambahan seperti pembagian partisi data yang lebih seimbang atau peningkatan spesifikasi perangkat keras.

C. Pengaruh Nilai K terhadap Akurasi dan Waktu Eksekusi

Pengujian ketiga dilakukan dengan N=6 core dan data latih 300.000 yang tetap, kemudian memvariasikan nilai K dari 1 hingga 9. Hasil pengujian disajikan pada Tabel 3.

Tabel 3. Hasil Pengujian Variasi Nilai K (N=6, Data=300.000)

Nilai K	Akurasi (%)
1	96,11
3	96,49
5	96,67
7	96,78
9	96,82



Berdasarkan Tabel 3, perubahan nilai K berdampak pada akurasi klasifikasi model. Akurasi terendah tercatat pada K=1 sebesar 96,11%, kemudian meningkat secara konsisten pada K=3 menjadi 96,49%, K=5 sebesar 96,67%, K=7 sebesar 96,78%, hingga mencapai akurasi tertinggi sebesar 96,82% pada K=9. Berbeda dengan percobaan sebelumnya, akurasi pada pengujian ini tidak menunjukkan fluktuasi melainkan meningkat secara stabil di setiap penambahan nilai K. Hal ini menunjukkan bahwa menambah

jumlah tetangga referensi secara konsisten menghasilkan prediksi yang lebih stabil dan akurat, karena keputusan klasifikasi didasarkan pada lebih banyak titik data di sekitarnya. Perlu dicatat bahwa selisih akurasi antara nilai K terkecil dan terbesar hanya sebesar 0,71%, yang mengindikasikan bahwa model tetap konsisten dan tidak terlalu sensitif terhadap perubahan nilai K dalam rentang pengujian ini.

V. KESIMPULAN DAN SARAN

Berdasarkan hasil eksperimen dan pembahasan yang telah dilakukan, dapat ditarik beberapa kesimpulan. Pertama, implementasi komputasi paralel dengan MPI pada algoritma KNN terbukti efektif meningkatkan efisiensi waktu eksekusi dalam mengklasifikasikan performa akademik siswa. Penambahan jumlah core mampu menurunkan waktu eksekusi secara signifikan, dengan waktu terbaik dicapai pada $N=3$ sebesar 224,84 detik dibandingkan $N=1$ yang membutuhkan 330,62 detik. Namun, penambahan core melampaui titik optimal justru meningkatkan waktu eksekusi akibat overhead komunikasi antar proses. Kedua, paralelisasi dengan MPI berhasil menjaga stabilitas akurasi klasifikasi, dengan akurasi tertinggi sebesar 96,82% pada $K=9$, menunjukkan bahwa distribusi beban kerja ke beberapa core tidak mengganggu kinerja prediksi. Ketiga, peningkatan volume data latih berbanding lurus dengan waktu eksekusi, di mana waktu meningkat dari 48,36 detik pada 100.000 data hingga 224,81 detik pada 300.000 data, karena beban komputasi pada setiap core bertambah seiring besarnya data yang diproses. Keempat, variasi nilai K menunjukkan tren akurasi yang meningkat secara konsisten dari 96,11% pada $K=1$ hingga 96,82% pada $K=9$, dengan selisih yang relatif kecil sebesar 0,71%, mengindikasikan bahwa model tidak terlalu sensitif terhadap perubahan nilai K.

Untuk penelitian selanjutnya, disarankan agar dilakukan pengujian pada dataset yang lebih besar dengan menggunakan infrastruktur komputasi terdistribusi multi-node untuk mengoptimalkan potensi MPI. Selain itu, perlu dipertimbangkan strategi pembagian partisi data yang lebih seimbang guna meminimalkan overhead komunikasi antar proses dan meningkatkan efisiensi sistem secara keseluruhan.

VI. UCAPAN TERIMA KASIH (OPTIONAL)

Penulis mengucapkan terima kasih kepada Bapak/Ibu Dosen Program Studi Teknik Informatika, Fakultas Teknik, Universitas Mataram yang telah memberikan bimbingan,

arahan, dan masukan selama proses penelitian dan penulisan makalah ini berlangsung. Penulis juga menyampaikan terima kasih kepada rekan-rekan mahasiswa yang telah memberikan dukungan dan diskusi yang konstruktif. Tidak lupa, penulis mengucapkan terima kasih kepada pihak penyelenggara SENIFORMA 2026, Seminar Nasional Informatika Universitas Mataram, yang telah menyediakan wadah bagi penulis untuk mempublikasikan hasil penelitian ini.

DAFTAR PUSTAKA

Penulisan daftar pustaka sepenuhnya mengikuti sistem IEEE di mana daftar pustaka diberi nomor dalam kurung siku [2] berdasarkan urutan kutipan dalam teks. Referensi yang digunakan pada daftar pustaka sebaiknya dari jurnal atau prosiding konferensi terkini. Contoh penulisan daftar pustaka adalah sebagai berikut:

- [1] F. Tatimu, V. R. Palilingan, dan I. Rianto, "Analisis Klasifikasi Kelulusan Mahasiswa menggunakan Algoritma K-Nearest Neighbor di Fakultas Teknik Universitas Negeri Manado," *Journal of Education Method and Technology*, vol. 3, 2020.
- [2] D. Setiyadi dan A. Nurdin, "Data Mining Potensi Akademik Siswa Berbasis Online," vol. 2, no. 1, 2012.
- [3] M. R. Arrohman, N. Khoiriyah, A. Fawaida, dan D. L. Fithri, "Analisis Klasifikasi Kelulusan Siswa Menggunakan Metode KNN (K-Nearest Neighbor) Random Forest pada Dataset Students Performance," *Seminar Nasional Teknologi Informasi*, 2023.
- [4] T. Hidayat, Y. Handayani, dan A. Syaifudin, "Implementasi Algoritma K-Nearest Neighbor," *Jurnal Ilmiah Rekayasa dan Manajemen Sistem Informasi*, vol. 9, no. 2, hlm. 118–124, 2023.
- [5] A. N. Hilmi, E. Y. Puspaningrum, dan H. E. Wahanani, "Implementasi Algoritma K-Nearest Neighbor (KNN) untuk Identifikasi Penyakit pada Tanaman Jeruk Berdasarkan Citra Daun," *Router: Jurnal Teknik Informatika dan Terapan*, vol. 2, no. 2, hlm. 107–117, 2024.
- [6] R. Dewi dan A. Sirait, "Implementasi Algoritma K-Nearest Neighbor (KNN) untuk Klasifikasi Tingkat Literasi Siswa," *Jurnal Teknik Informatika dan Teknologi Informasi*, vol. 5, no. 3, hlm. 56–70, 2025.
- [7] A. Kurniati dan A. Hakim, "Analisis kemiskinan multidimensi di Kota Magelang

dengan metode Multidimensional Poverty Index (MPI)," *Jurnal Kebijakan Ekonomi dan Keuangan*, hlm. 237–246, 2026.

- [8] S. Suliman, "Implementasi Data Mining Terhadap Prestasi Belajar Mahasiswa Berdasarkan Pergaulan dan Sosial Ekonomi Dengan Algoritma K-Means Clustering," *SIMKOM*, vol. 6, no. 1, hlm. 1–11, 2021.
- [9] D. Puspita Sari, S. Shofia Hilabi, dan A. Hananto, "Penerapan Data Mining Metode K-Nearest Neighbor Untuk Memprediksi Kelulusan Siswa Sekolah Menengah Pertama," *SMARTICS Journal*, vol. 9, no. 1, hlm. 14–19, 2023.
- [10] M. A. Wardana dan S. Suherman, "Penerapan Data Mining Pengelompokan Data Penjualan Motor di PT. Nusantara Surya Sakti Soppeng Menggunakan Metode K-Means Clustering," *JISTI*, vol. 8, no. 1, hlm. 123–132, 2025.
- [11] A. De Wibowo Muhammad Sidik, I. Himawan Kusumah, A. Suryana, M. Artiyasa, dan A. P. Junfithrana, "Gambaran Umum Metode Klasifikasi Data Mining," vol. 2, no. 2, hlm. 34–38, 2020.
- [12] J. Homepage dan B. Prasetyo, "Implementation of K-Nearest Neighbor Classification Algorithm as Decision Support Method in Staple Food Aid Distribution to Society," *IJIRSE: Indonesian Journal of Informatic Research and Software Engineering*, 2022.
- [13] A. Khairi, I. Fahrezi, I. Sahputra, dan F. Anshari, "Implementation of K-Nearest Neighbor (KNN) Method to Determine House Locations in Batuphat and Tambon Tunong Areas, Aceh," *Jurnal Bisnis Informatika*, vol. 10, no. 1, 2021.
- [14] R. S. Daulay, "Analisis Kritis dan Pengembangan Algoritma K-Nearest Neighbor (KNN): Sebuah Tinjauan Literatur," *Jurnal Pendidikan Sains dan Komputer*, vol. 4, no. 02, hlm. 131–141, 2024.
- [15] P. I. Sijabat, E. Br Barus, Y. Laora Slingga, dan D. A. Putri, "Penerapan Algoritma K-Nearest Neighbor (KNN) untuk Menganalisis Pendapat Siswa terhadap Edukasi Anti Perundungan," *Jurnal Sistem Informasi TGD*, vol. 5, no. 1, hlm. 217–222, 2026.
- [16] H. Said, N. Matondang, H. Nurramdhani Irmanda, dan S. Informasi, "Penerapan Algoritma K-Nearest Neighbor untuk Memprediksi Kualitas Air yang Dapat Dikonsumsi," vol. 21, no. 2, 2022.
- [17] A. Lesmana dkk., "Komputasi Paralel untuk Pengolahan Prestasi Akademik Mahasiswa," 2017.
- [18] I. N. Sofiyanto, A. B. Saputra, R. F. Rakhmadianto, H. Nur, E. Febrianto, A. M. Endiwan, dan A. A. Sari, "Prediksi Kelulusan Mahasiswa Menggunakan Algoritma K-Nearest Neighbor (KNN) dengan Klasifikasi Biner," dalam *Prosiding Seminar Nasional Teknologi Informasi dan Bisnis (SENATIB)*, 2025.